



Title:

Assignment 1 — Search Strategies

CS 3081-2 | Fall 2026

Student name and ID:

Reema Abualnaja S24209805

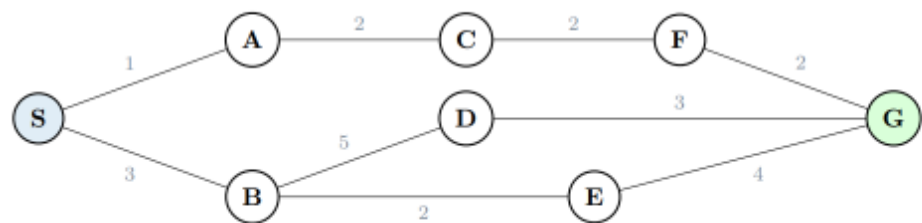
Instructor: Dr. Naila Marir

Instructions. Answer all four questions. Show your working — a correct final answer with no reasoning earns partial credit only. Where a question asks for the order in which nodes are expanded, write the nodes as a sequence, e.g. *S, A, C, G*.

- When two nodes have equal priority, break ties **alphabetically**.
- A node is *expanded* when it is removed from the frontier and its successors are generated. Counting the goal test, a node is expanded even if it turns out to be the goal.
- Assume the **graph-search** form of each algorithm: never expand the same state twice.
- This is an individual assignment. You may discuss ideas, but the work you submit must be your own.

1 Uninformed search: BFS and DFS

Consider the following state graph. The start state is *S* and the goal state is *G*. Edges are undirected, and the number on each edge is its step cost.



For this question ignore the edge costs. BFS and DFS are *uninformed*: they do not look at cost or at any estimate of distance to the goal.

- (a) **Breadth-first search** [8 marks]
- List the order in which states are expanded by BFS, and give the path it returns.

Breadth-First Search | Queue (FIFO)

Step	Expand	Push	Frontier
1	S	A, B	[A, B]
2	A	C	[B, C]
3	B	D, E	[C, D, E]
4	C	F	[D, E, F]
5	D	G	[E, F, G]
6	E	-	[F, G]
7	F	-	[G]
8	G	-	-

Goal *G* is reached at step 5, expanded at step 8.

Order of expansion: *S, A, B, C, D, E, F, G*

Returned path: *S → B → D → G*

(b) Depth-first search [8 marks]

List the order in which states are expanded by DFS, and give the path it returns. Assume successors are pushed onto the stack in reverse alphabetical order, so that the alphabetically first successor is expanded next.

Depth-First Search | Stack (LIFO)

Step	Expand	Push	Frontier
1	S	B, A	[B, A]
2	A	C	[B, C]
3	C	F	[B, F]
4	F	G	[B, G]
5	G	-	[B]

Goal G is reached at step 4 and expanded at step 5.

Order of expansion: S, A, C, F, G

Returned path: $S \rightarrow A \rightarrow C \rightarrow F \rightarrow G$

(c) Comparison [6 marks]

BFS and DFS returned different paths. State which path is shorter in number of steps, and explain why BFS is guaranteed to find that path while DFS is not.

$S \rightarrow B \rightarrow D \rightarrow G = 3$ steps

$S \rightarrow A \rightarrow C \rightarrow F \rightarrow G = 4$ steps

Breadth-First Search returns the shorter path because it checks the nodes level by level. While Depth-First Search always expands the deepest node first, which can lead to a longer path.

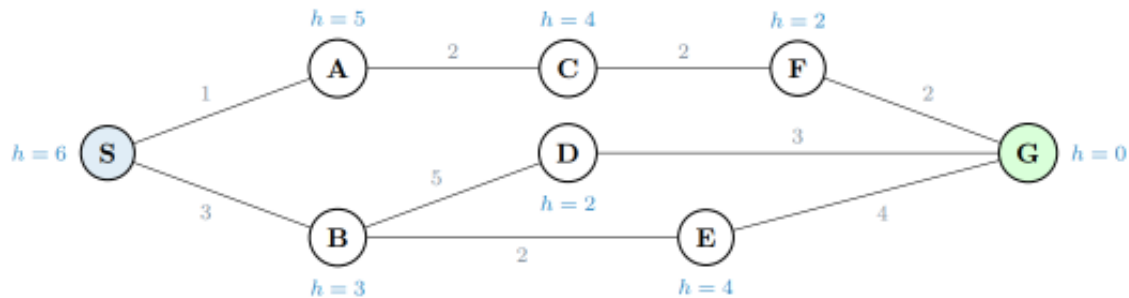
(d) Frontier data structures [3 marks]

BFS and DFS differ by one design choice. Name the data structure each uses for its frontier, and explain in one sentence how that choice produces the difference in behaviour you observed.

BFS expands the shallowest node in the frontier, so it uses a queue (first-in, first-out data type). DFS expands the deepest node in the frontier, so it uses a stack (last-in, first-out data type).

2 Informed search: Greedy Best-First and A*

The same graph is repeated below. Each state is now annotated with a heuristic value $h(n)$ — an estimate of the cost from n to the goal G .



(a) Greedy best-first search

[8 marks]

Greedy best-first expands the frontier node with the smallest $h(n)$. List the order of expansion and the path returned.

Greedy best-first Search

Node	S	A	B	C	D	E	F	G
$h(n)$	6	5	3	4	2	2	2	0

Step	Expand	Push	Frontier
1	S	A (5), B (3)	[B(3), A (5)]
2	B	C (4), D (4)	[D (2), E(4), A (5)]
3	D	F (3)	[G (0), E (4), A (5)]
4	G	G (0)	[E (4), A (5)]

Greedy chooses the node with the smallest $h(n)$, if two nodes have the same $h(n)$ it chooses them in alphabetical order.

Goal G is found at step 3, and expanded at step 4

Order of expansion: S, B, D, G

Returned path: $S \rightarrow B \rightarrow D \rightarrow G$

(b) A* search

[12 marks]

A* expands the frontier node with the smallest $f(n) = g(n) + h(n)$. Complete the table for every node *at the moment it is expanded*, then give the order of expansion and the path returned.

Order	Node	$g(n)$	$h(n)$	$f(n) = g(n) + h(n)$
1	S	0	6	6
2	_____	_____	_____	_____
3	_____	_____	_____	_____
4	_____	_____	_____	_____
5	_____	_____	_____	_____
6	_____	_____	_____	_____

Order of expansion: _____

Path returned: _____ Total cost: _____

A* Search

Order	Node	$g(n)$	$h(n)$	$F(n) = g(n) + h(n)$
1	S	0	6	6
2	A	1	5	6
3	B	3	3	6
4	C	3	4	7
5	F	5	2	7
6	G	7	0	7

Order of expansion: S, A, B, C, F, G

Returned path: $S \rightarrow A \rightarrow C \rightarrow F \rightarrow G$

Total cost: 7

A* chooses the node with the smallest $f(n) = g(n) + h(n)$. When nodes have the same $f(n)$, alphabetical order is used.

(c) Why they differ

[6 marks]

Greedy best-first and A* returned different paths, or explored a different number of nodes, or both. Explain the difference by referring to what each algorithm ignores.

Greedy Best-First Search chooses the node based only on $h(n)$, while A* considers both the cost $g(n)$ and the estimated cost $h(n)$. That's the reason A* chooses a different path and finds a cheaper path with cost 7 instead of Greedy's cost 11.

3 Admissibility and optimality

(a) Definition

[5 marks]

Define an **admissible** heuristic. Your definition must be precise enough that someone could check a given heuristic against it.

Admissible: It never overestimates the true cost. It can underestimate the cost or be equal to the true cost.

(b) Testing the heuristic

[10 marks]

For the graph in Question 2, compute the true cost $h^*(n)$ of the cheapest path from each node to G , and decide whether the given h is admissible.

Node	S	A	B	C	D	E	F	G
$h(n)$ given	6	5	3	4	2	4	2	0
$h^*(n)$ true	_____	_____	_____	_____	_____	_____	_____	_____
$h \leq h^*$?	_____	_____	_____	_____	_____	_____	_____	_____

Is h admissible? _____

If not, name every node that violates the condition: _____

Node	S	A	B	C	D	E	F	G
$h(n)$ given	6	5	3	4	2	4	2	0
$h^*(n)$ true	7	6	6	4	3	4	2	0
$h \leq h^*$?	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes

Is h admissible? Yes.

If not, name every node that violates the condition: None

(c) Consequence

[6 marks]

Suppose a heuristic *overestimates* the true cost at exactly one node. Explain what this can do to the solution A^* returns, and why uniform-cost search (A^* with no heuristic) could never suffer from this problem.

If the heuristic overestimates the true cost, A^* may choose the wrong path and return a more expensive solution. Uniform-Cost Search doesn't use heuristics, so it is not affected by heuristic overestimation.

(d) A perfect heuristic

[3 marks]

Suppose $h(n) = h^*(n)$ for every node. How does A* behave in this case? Justify your answer in one sentence.

If $h(n) = h(n)^*$ then the heuristic gives the exact cheapest cost remaining from every node to the goal.

perfect heuristic, A* knows the exact remaining cost, $f(n) = g(n) + h(n)^*$ can give us the exact total cost of the solution through that node.

4 Comparing the four strategies

(a) Properties table

[8 marks]

Complete the table. Name the data structure each strategy uses for its frontier. For *complete* and *optimal*, write Yes or No and state any condition required for that answer to hold.

Strategy	Frontier	Complete?	Optimal?
Breadth-first	Queue (FIFO)	Yes	Yes, if all step costs are equal
Depth-first	Stack (LIFO)	Yes, Under graph-search on a finite graph	No
Greedy best-first	Priority queue using $h(n)$	Yes, Under graph-search on a finite graph	No, in general
A*	Priority queue using $f(n) = g(n) + h(n)$	Yes, Under standard positive-cost conditions	Yes, with an admissible heuristic

(b) Choosing a strategy

[13 marks]

For each scenario, name the most appropriate of the four strategies and justify your choice. Refer to the properties in part (a) — a bare name earns no marks.

- i. A delivery drone must find the cheapest flight path between two warehouses, and the straight-line distance to the destination is known for every waypoint.

A* - It considers the cost of the route and uses the straight-line distance as a heuristic to estimate the remaining distance, which makes it suitable to find a cheaper flight path.

- ii. A web crawler must find *any* page containing a keyword. The link graph is very deep, memory is tight, and the shortest link path is not important.

DFS - It uses less memory and can explore a very deep graph without storing a large frontier. The reason that DFS is suitable in this situation is it finds any page containing the keyword, without needing the shortest path.

- iii. A game AI must reach a target square quickly on a large map. A good distance estimate exists, but the move does not need to be the cheapest — only fast to compute.

Greedy Best-First Search – Is suitable for this scenario because it uses the heuristic to choose the node that is estimated to be close to the target. The goal is to reach the target quickly, so it doesn't need the route to be the cheapest.

(c) Short answer

[4 marks]

BFS is complete and optimal for uniform step costs, but is often impractical on large problems. State the main reason, and name one property of the search problem that makes it worse.

BFS can be impractical for large problems because it stores a lot of nodes in its frontier. What can make the problem worse is that the expanded node can generate many nodes which can affect the frontier to grow very quickly.